

新一代商用密码算法 ARM 架构 实现自评估指引

商用密码标准研究院

2026 年 6 月

目 录

一、算法实现须知.....	1
1.1 编写目的.....	1
1.2 适用范围.....	1
1.3 评估指引.....	2
1.4 材料准备.....	2
二、算法实现指引.....	2
2.1 软硬件环境.....	2
2.2 算法实现.....	3
三、实现自评估指标.....	4
3.1 指标定义.....	4
3.2 算法实现版本.....	5
3.3 自评估测试.....	7
3.4 自评估测试数据.....	7
3.5 自评估测试报告.....	9
四、其他事项.....	9

一、算法实现须知

1.1 编写目的

为规范新一代公钥密码算法(含密钥封装、数字签名、密钥交换)和密码杂凑算法在 ARM 架构下的算法实现自评估流程,明确自评估指标和要求,达到算法实现的正确性、可比性和高效性的目的,特制定本指引。

本指引基于 ARM 架构特性,为算法提交者提供 ARM 架构参考实现版和优化实现版的技术依据和操作指引,不单独构成新增的实现要求。算法提交者可依据算法提交要求,提交相应的参考实现版和优化实现版(包括性能优化版和资源优化版)。

自评估结果作为算法提交材料中的可复核技术证据,仅用于说明实现环境、测试过程和测试结果。

1.2 适用范围

本指引适用于公钥密码算法和密码杂凑算法共两类算法在 ARM 架构下的实现自评估,包括参考实现版和优化实现版,其中优化实现版可进一步分为性能优化版和资源优化版。各版本说明如下:

(1) **参考实现版**: 注重逻辑正确性、实现简洁性与易开发性,不要求额外的性能/资源优化设计,不引入特定优化手段,便于理解算法实现机理。

(2) **性能优化版**: 以算法实现的性能最大化为核心目标,可通过 ARM 扩展指令集特性等优化手段降低时延、提升吞吐率,允许适度增加存储资源占用,以适配高性能、高吞吐的软件应用场景。

(3) **资源优化版**: 以算法实现的存储资源占用最小化为核心目标,可通过精简数据结构、复用存储资源、简化运算过程等手段减少存储资源占用,以适配资源受限的软件应用场景。

1.3 评估指引

新一代商用密码算法的 ARM 架构实现自评估指引如下：

（1）**要求统一**：公钥密码算法和密码杂凑算法的自评估应基于第三章提供的实现自评估开展。

（2）**分类提交**：应按算法类别和实现版本分别提交。其中，公钥密码算法区分密钥封装、数字签名、密钥交换三种功能，并体现不同版本的实现差异。

（3）**指标全覆盖**：不同实现版本的评估结果应覆盖功能、性能、资源消耗、传输与存储开销等指标。

（4）**真实可复现**：实现自评估测试报告中的运行结果应真实且可复现。

1.4 材料准备

（1）算法提交者应按算法类别和实现版本分别整理材料，并整体打包提交。每一版至少包含以下材料：

序号	材料名称	补充说明
1	算法源代码	完整的算法实现源代码，包括编译配置文件、实现说明文档等。
2	自评估代码和数据	调用算法实现源代码、用于实现功能、性能、资源消耗、传输与存储开销等自评估操作的代码和数据。
3	自评估报告	测试结果。
4	依赖说明	第三方库依赖清单及版本信息等。

（2）材料命名方式：材料文件命名应采用“算法类别-算法名-ARM-实现版本”，如“密钥封装-SM2-ARM-资源优化版”等。

二、算法实现指引

2.1 软硬件环境

为确保自评估结果的可比性，ARM 架构实现建议采用以下基准

环境配置。

硬件环境采用如下配置：

序号	组件	配置说明
1	CPU 型号	型号无要求。64 位 ARM 架构，兼容 ARM v8.2-A 指令集。
2	主频	建议大于 2 GHz。
3	ARM 指令集	ARM v8.2-A 标准指令集，支持 NEON 和 SVE 指令。
4	核心数	只使用单核开展 ARM 架构的测试。
5	内存	建议大于 4GB。

软件环境采用如下配置：

序号	组件	配置说明
1	操作系统	Linux 操作系统，报告中应说明实际使用的发行版名称、版本，建议内核 4.19 及以上版本。
2	编译器	GNU GCC 8.5.0 及以上版本。
3	构建工具	CMake 3.11.4 及以上版本。

2.2 算法实现

算法的 ARM 架构实现指引如下：

（1）**接口规范**：两类算法应分别按照《公钥密码算法提交要求》和《密码杂凑算法提交要求》指定的编程接口进行实现。

（2）**代码规范**：算法实现代码应采用 ISO/IEC 9899:1999（C99）语言实现。其中性能优化版和资源优化版可根据需要使用面向 ARM 架构的内联汇编或独立汇编源文件。同时，算法实现代码清晰、规范，注释完整，避免冗余代码、死循环、缓冲区溢出等常见漏洞。

三、实现自评估指标

3.1 指标定义

本次自评估内容划分为基础核查项、备案信息项、量化对比指标三大类别。基础核查项包含接口符合性、KAT 校验正确性、程序可构建性、测试脚本可复现性以及提交材料完整性；备案信息项涵盖第三方依赖情况、随机数生成方法、编译参数、目标指令集、优化方式、实际测试环境等内容；量化对比指标包含时钟周期数、运算吞吐量、内存占用、数据尺寸、交互轮数等维度。基础核查项与备案信息项，主要用于判定评估结果真实有效，确保数据可复核校验；量化对比指标，则支撑后续开展性能表现、资源消耗、数据传输及存储开销等方面的分析工作。

实现自评估指标定义如下：

一级指标	二级指标	说明
功能指标	正确性	使用已知答案测试向量验证算法功能是否正确实现。
性能指标	时钟周期数	完成一次算法核心操作（密钥生成、签名、验证、封装、解封装、建立共享秘密密钥、生成杂凑值）所需的 CPU 时钟周期数，单位 cycles。
	运算吞吐量	公钥密码算法：单位时间内完成的算法操作次数，包括密钥封装、数字签名、密钥交换，单位 ops/s； 密码杂凑算法：单位时间内处理的运算数据字节数，单位 MB/s。
资源消耗指标	静态内存占用	算法加载后未执行运算时的基础内存占用（代码段、常量、全局变量等），单位 Bytes。

	峰值内存占用	算法运行过程中观测到的最大内存占用(含堆、栈使用量运行时的资源以及代码段、常量、全局变量等静态资源)，单位 Bytes。
传输与存储开销指标	数据尺寸	公钥密码算法：公钥、私钥、签名、密文、密钥交换消息等数据尺寸，单位 Bytes。
	轮数	公钥密码算法：密钥交换的交互轮数等。

3.2 算法实现版本

本指引为算法提交者提供基于ARM架构参考实现版、性能优化版和资源优化版共三个版本的算法实现指引。此外，鼓励提交面向特定应用场景、具有实践价值的其他版本。

3.2.1 参考实现版

参考实现版使用纯C语言编写，遵循ISO C99标准，避免使用汇编语言、编译器特定扩展和硬件特定指令集等。

参考实现版应采用如下的编译配置：

gcc -std=c99 -Wpedantic -Wall -Wextra -O2。其中：

- -std=c99：指定编译器遵循 ISO C99 标准，确保语法规范的一致性与跨平台可移植性；
- -Wpedantic：强制要求代码严格符合 ISO C99 标准，禁止使用非标准的编译器专有扩展语法；
- -Wall：开启编译器最常用的警告集合，用于识别并修复代码中明显的逻辑隐患；
- -Wextra：在基础警告之上开启额外的严格检查，捕捉更隐蔽的代码缺陷；
- -O2：执行中度性能优化，在不显著增加程序体积的情况下提升代码执行效率。

对于所依赖的第三方库，建议使用相同的编译配置。

3.2.2 性能优化版

性能优化版实现可使用 C 语言、面向 ARM 架构的内联汇编或独立汇编源文件，以实现算法性能最大化为目标。可充分利用 ARM 架构的如下特性：

- SIMD 向量化：如 NEON（128-bit）、SVE（可伸缩向量，128-2048bit）；
- 工程实现优化：如循环展开、流水线化、内联函数、-O3 编译选项等。

性能优化版应采用如下的编译配置：

```
gcc -O3 -march=armv8.2a+sve -flto -fomit-frame-pointer -std=c99  
-Wpedantic -Wall -Wextra。
```

- -O3: 开启高级别性能优化，在-O2 的基础上增加自动向量化、循环展开及更激进的函数内联，旨在最大化程序的执行速度；
- -march=armv8.2a+sve: 指定目标指令集架构为 ARMv8.2-A，开启 ARM 的可变长度向量扩展（SVE）；
- -flto: 启用链接时优化，允许编译器跨源文件进行全局分析和优化，从而进一步缩减代码体积并提升性能；
- -fomit-frame-pointer: 在不必要的情况下不将栈帧指针存储在寄存器中，从而释放通用寄存器供程序逻辑使用。

对于所依赖的第三方库，建议使用相同的编译配置。

3.2.3 资源优化版

资源优化版实现可使用 C 语言、面向 ARM 架构的内联汇编或独立汇编源文件，以降低存储资源占用为目标。可采用流式处理替代一次性加载全量数据、及时释放无用内存、优化数据结构、复用内存减少创建销毁代价等技术编写代码达到优化目标。

资源优化版应采用如下的编译配置：

```
gcc -Os -march=armv8-a -flto -fomit-frame-pointer -std=c99  
-Wpedantic -Wall -Wextra。其中：
```

- -Os: 开启尺寸优化, 以减小代码体积为目标;
- -march=armv8-a: 指定目标指令集架构为 ARMv8-A;
- -flto: 启用链接时优化, 允许编译器跨源文件进行全局分析和优化, 从而进一步缩减代码体积并提升性能;
- -fomit-frame-pointer: 在不必要的情况下不将栈帧指针存储在寄存器中, 从而释放通用寄存器供程序逻辑使用。

对于所依赖的第三方库, 建议使用相同的编译配置。

3.3 自评估测试

算法 ARM 架构实现自评估测试, 应包括如下事项:

(1) **功能测试**: 使用已知答案测试 (KAT) 的测试向量输入算法, 通过比对算法的实际输出与标准已知输出是否一致, 从而验证算法功能是否正确实现。

(2) **性能测试**: 多次执行, 利用运行时间和数据处理量计算性能指标。

(3) **资源消耗测试**: 多次执行, 测量并统计静态内存占用和峰值内存占用。

(4) **传输与存储开销测试**: 测量并统计公钥密码算法的公钥、私钥、签名、密文、密钥交换消息等数据尺寸。

自评估测试应覆盖算法全部实例、全部声明安全强度等级和全部指定编程接口。

注: 需要多次执行并统计的场景, 测试次数应大于等于100次。

3.4 自评估测试数据

功能测试的全部测试向量应记录在报告中。性能测试和资源消耗测试使用可复现的输入, 并在测试报告中说明输入数据来源或生成方式。

针对需要随机数的测试, 采用固定测试集, 或记录随机种子和生成规则, 以达到结果可复现的目的。随机数生成规则推荐使用算法提

交要求所提供的编程接口文件中给出的方法。

3.4.1 公钥密码算法测试数据

针对密钥封装功能，准备密钥生成、封装、解封装的功能测试数据，并记录公钥、私钥、密文和封装密钥的尺寸数据。

针对数字签名功能，准备密钥生成、签名、验签的功能测试数据，并记录公钥、私钥、签名的尺寸数据。

针对密钥交换功能，准备初始化、消息生成、共享秘密密钥派生的功能测试数据，并记录消息长度、交互轮数、长期公钥（如有）、长期私钥（如有）和共享秘密密钥的尺寸数据。

公钥密码算法测试数据按数字签名、密钥封装、密钥交换三类功能准备，针对不同安全强度等级，分别给出性能测试结果。

功能	指标	备注
密钥封装	密钥生成性能、封装性能、解封装性能	只测试该功能的性能指标,不考虑封装密钥长度差异影响。
数字签名	密钥生成性能、签名性能、验签性能	使用 64 字节数据作为待签名消息输入进行测试,避免杂凑计算开销差异对测试结果可比性的影响。
密钥交换	密钥交换性能	不计算网络传输的用时。

3.4.2 密码杂凑算法测试数据

针对密码杂凑算法，准备计算杂凑值的功能测试数据；若算法支持可扩展输出或变长输出，建议覆盖不同输出长度下的功能测试。性能测试建议在 S1-S8 输入规模下分别统计时钟周期数和运算吞吐量。

密码杂凑算法选择 S1-S8 共 8 级数据作为输入的性能指标，数据从字节级至 KB 级，具体分级如下：

级别	数据长度
S1	32Bytes
S2	128Bytes
S3	512Bytes

S4	1024Bytes
S5	4096Bytes
S6	8192Bytes
S7	16384Bytes
S8	65536Bytes

3.5 自评估测试报告

算法设计者完成自评估测试后，应编制包含以下内容的自评估测试报告：

（1）**算法基本信息**：算法类别、算法名称、细分功能、实现版本、参数集/安全级别。

（2）**评估环境信息**：运行环境，包含处理器型号、主频、内存、操作系统发行版及内核版本、编译器版本、构建工具版本，及开始时间、结束时间等。

（3）**功能测试**：每个测试用例的测试结果。

（4）**性能测试**：平均耗时、平均周期数。

（5）**资源消耗测试**：峰值内存、静态内存等。

（6）**传输与存储开销测试**：公钥密码算法的公钥、私钥、签名、密文、密钥交换消息等数据尺寸以及密钥交换轮数等。

（7）**原始证据索引**：列明各项汇总指标对应的测试命令、输入数据、原始日志、结果 CSV/JSON 文件和计算脚本。

测试结果清晰、明确、无歧义，并具备可复核性。

四、其他事项

算法提交要求和编程接口请查阅商用密码标准研究院网站 www.niccs.org.cn。以下平台已提供自评估代码和数据，可免费使用：

https://gitcode.com/openHiTLS/ngcc_bench 或

https://github.com/openHiTLS/ngcc_bench。源代码目录结构可参考上述平台。请注意以上平台与本次征集活动无关，与商用密码标准研究

院无关联关系，该平台的使用非强制要求，请自行处理密码算法相关知识产权的保护工作。